BẢN CHUYÊN SÂU • 2026

BẢN ĐỒ 20 NGHỀ CNTT

LỘ TRÌNH TỪ CON SỐ 0

Tìm nghề phù hợp • Hiểu công việc thực tế • Học đúng kỹ năng • Làm dự án • Xây portfolio • Chuẩn bị ứng tuyển

20 NGHỀ

Hướng đi đa dạng

12 THÁNG

Lộ trình mẫu

EBOOK ĐỊNH HƯỚNG & THỰC HÀNH

00 / Cách sử dụng tài liệu

Đọc có chiến lược, thực hành có bằng chứng

01 Khám phá nghề

Đọc trang tổng quan của 3–5 nghề. So sánh công việc hằng ngày, kỹ năng cốt lõi và sản phẩm cần tạo.

02 Chọn một hướng chính

Không học đồng thời quá nhiều stack. Chọn một nghề để thử nghiêm túc trong 6–8 tuần, sau đó đánh giá mức phù hợp.

03 Học theo đầu ra

Mỗi giai đoạn cần có bằng chứng: mã nguồn, bản demo, test report, dashboard, prototype hoặc tài liệu phân tích.

04 Điều chỉnh tiến độ

Lộ trình 12 tháng là tham khảo cho người học đều đặn 8–12 giờ/tuần. Nhanh hoặc chậm hơn đều bình thường.

05 Dùng AI có kiểm chứng

Có thể dùng AI để giải thích khái niệm, gợi ý bài tập và review code; tự kiểm tra kết quả, bảo mật dữ liệu và ghi rõ đóng góp của mình.

MỤC LỤC • 20 CHUYÊN ĐỀ

01. Frontend Developer
02. Backend Developer
03. Full-stack Developer
04. Mobile Developer
05. Game Developer
06. QA Manual Tester
07. Automation QA Engineer
08. Business Analyst IT
09. UI/UX Designer
10. Data Analyst
11. Data Engineer
12. Data Scientist
13. AI/ML Engineer
14. Generative AI Engineer
15. Cybersecurity Analyst
16. DevOps Engineer
17. Cloud Engineer
18. System Administrator
19. Embedded/IoT Engineer
20. Product Manager (Tech)

01 / Frontend Developer

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Frontend Developer tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng HTML, CSS, JavaScript, TypeScript, React. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

HTML, CSS, JavaScript, TypeScript, React; DOM, bất đồng bộ, accessibility, state, API, tối ưu Core Web Vitals.

CÔNG CỤ CẦN LÀM QUEN

VS Code, Git, Chrome DevTools, Vite. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- Frontend Intern, Junior Frontend Developer
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

01 / Frontend Developer

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 DOM

Học khái niệm và cách sử dụng DOM trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 BẤT ĐỒNG BỘ

Học khái niệm và cách sử dụng bất đồng bộ trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 ACCESSIBILITY

Học khái niệm và cách sử dụng accessibility trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 STATE

Học khái niệm và cách sử dụng state trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 API

Học khái niệm và cách sử dụng API trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 TỐI ƯU CORE WEB VITALS

Học khái niệm và cách sử dụng tối ưu Core Web Vitals trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

01 / Frontend Developer

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | HTML/CSS

Học và ứng dụng HTML/CSS; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | JAVASCRIPT

Học và ứng dụng JavaScript; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | REACT

Học và ứng dụng React; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | TYPESCRIPT

Học và ứng dụng TypeScript; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | KIỂM THỬ VÀ TRIỂN KHAI

Học và ứng dụng kiểm thử và triển khai; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | PORTFOLIO VÀ ỨNG TUYỂN

Học và ứng dụng portfolio và ứng tuyển; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

01 / Frontend Developer

3 dự án tăng dần độ khó

DỰ ÁN 1 • TRANG GIỚI THIỆU KHÓA HỌC CÓ RESPONSIVE

Bài toán: Xây dựng trang giới thiệu khóa học có responsive với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • DASHBOARD QUẢN LÝ HỌC VIÊN

Bài toán: Xây dựng dashboard quản lý học viên với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • WEBSITE THƯƠNG MẠI ĐIỆN TỬ CÓ GIỎ HÀNG

Bài toán: Xây dựng website thương mại điện tử có giỏ hàng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

01 / Frontend Developer

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: DOM, bất đồng bộ, accessibility.
- Biết sử dụng các công cụ chính: VS Code, Git, Chrome DevTools.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Frontend Developer, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng HTML như thế nào trong dự án?
3. Khi DOM gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

02 / Backend Developer

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Backend Developer tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Python, SQL, HTTP, REST API, FastAPI. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Python, SQL, HTTP, REST API, FastAPI; Cơ sở dữ liệu, transaction, authentication, caching, bảo mật API.

CÔNG CỤ CẦN LÀM QUEN

Postman, PostgreSQL, Docker, Git. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- Backend Intern, Junior Backend Developer
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

02 / Backend Developer

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 CƠ SỞ DỮ LIỆU

Học khái niệm và cách sử dụng Cơ sở dữ liệu trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 TRANSACTION

Học khái niệm và cách sử dụng transaction trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 AUTHENTICATION

Học khái niệm và cách sử dụng authentication trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 CACHING

Học khái niệm và cách sử dụng caching trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 BẢO MẬT API

Học khái niệm và cách sử dụng bảo mật API trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 CƠ SỞ DỮ LIỆU

Học khái niệm và cách sử dụng Cơ sở dữ liệu trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 / Backend Developer

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | PYTHON

Học và ứng dụng Python; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | SQL

Học và ứng dụng SQL; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | FASTAPI

Học và ứng dụng FastAPI; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | XÁC THỰC

Học và ứng dụng xác thực; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | DOCKER

Học và ứng dụng Docker; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | TRIỂN KHAI

Học và ứng dụng triển khai; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

02 / Backend Developer

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • API QUẢN LÝ THƯ VIỆN

Bài toán: Xây dựng api quản lý thư viện với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • HỆ THỐNG XÁC THỰC NGƯỜI DÙNG

Bài toán: Xây dựng hệ thống xác thực người dùng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • API ĐẶT LỊCH CÓ PHÂN QUYỀN

Bài toán: Xây dựng api đặt lịch có phân quyền với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

02 / Backend Developer

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: Cơ sở dữ liệu, transaction, authentication.
- Biết sử dụng các công cụ chính: Postman, PostgreSQL, Docker.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Backend Developer, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Python như thế nào trong dự án?
3. Khi Cơ sở dữ liệu gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

03 / Full-stack Developer

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Full-stack Developer tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng JavaScript, TypeScript, React, Node.js, SQL. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

JavaScript, TypeScript, React, Node.js, SQL; Kiến trúc client-server, CRUD, auth, deployment, logging.

CÔNG CỤ CẦN LÀM QUEN

VS Code, GitHub, Postman, Docker. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- Full-stack Intern, Junior Full-stack Developer
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

03 / Full-stack Developer

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 KIẾN TRÚC CLIENT-SERVER

Học khái niệm và cách sử dụng Kiến trúc client-server trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 CRUD

Học khái niệm và cách sử dụng CRUD trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 AUTH

Học khái niệm và cách sử dụng auth trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 DEPLOYMENT

Học khái niệm và cách sử dụng deployment trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 LOGGING

Học khái niệm và cách sử dụng logging trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 KIẾN TRÚC CLIENT-SERVER

Học khái niệm và cách sử dụng Kiến trúc client-server trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 / Full-stack Developer

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | WEB CĂN BẢN

Học và ứng dụng Web căn bản; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | REACT

Học và ứng dụng React; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | NODE.JS

Học và ứng dụng Node.js; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | SQL

Học và ứng dụng SQL; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | TÍCH HỢP

Học và ứng dụng tích hợp; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | DEPLOY

Học và ứng dụng deploy; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

03 / Full-stack Developer

3 dự án tăng dần độ khó

DỰ ÁN 1 • ỨNG DỤNG GHI CHÚ ĐA NGƯỜI DÙNG

Bài toán: Xây dựng ứng dụng ghi chú đa người dùng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • WEBSITE TUYỂN DỤNG

Bài toán: Xây dựng website tuyển dụng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • ỨNG DỤNG QUẢN LÝ CÔNG VIỆC

Bài toán: Xây dựng ứng dụng quản lý công việc với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

03 / Full-stack Developer

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: Kiến trúc client-server, CRUD, auth.
- Biết sử dụng các công cụ chính: VS Code, GitHub, Postman.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Full-stack Developer, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng JavaScript như thế nào trong dự án?
3. Khi Kiến trúc client-server gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

04 / Mobile Developer

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Mobile Developer tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Dart, Flutter, REST API, SQLite. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Dart, Flutter, REST API, SQLite; Widget, navigation, state, local storage, API, phát hành ứng dụng.

CÔNG CỤ CẦN LÀM QUEN

Android Studio, Flutter SDK, Firebase, Git. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- Flutter Intern, Junior Mobile Developer
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

04 / Mobile Developer

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 WIDGET

Học khái niệm và cách sử dụng Widget trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 NAVIGATION

Học khái niệm và cách sử dụng navigation trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 STATE

Học khái niệm và cách sử dụng state trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 LOCAL STORAGE

Học khái niệm và cách sử dụng local storage trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 API

Học khái niệm và cách sử dụng API trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 PHÁT HÀNH ỨNG DỤNG

Học khái niệm và cách sử dụng phát hành ứng dụng trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 / Mobile Developer

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | DART

Học và ứng dụng Dart; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | FLUTTER UI

Học và ứng dụng Flutter UI; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | STATE

Học và ứng dụng state; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | API

Học và ứng dụng API; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | LƯU TRỮ

Học và ứng dụng lưu trữ; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | BUILD ỨNG DỤNG

Học và ứng dụng build ứng dụng; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

04 / Mobile Developer

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • ỨNG DỤNG CHI TIÊU

Bài toán: Xây dựng ứng dụng chi tiêu với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • APP HỌC TỪ VỰNG

Bài toán: Xây dựng app học từ vựng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • APP ĐẶT LỊCH DỊCH VỤ

Bài toán: Xây dựng app đặt lịch dịch vụ với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

04 / Mobile Developer

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: Widget, navigation, state.
- Biết sử dụng các công cụ chính: Android Studio, Flutter SDK, Firebase.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Mobile Developer, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Dart như thế nào trong dự án?
3. Khi Widget gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

05 / Game Developer

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Game Developer tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng C#, Unity, toán vector, vật lý cơ bản. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

C#, Unity, toán vector, vật lý cơ bản; Game loop, collision, animation, AI đơn giản, tối ưu FPS.

CÔNG CỤ CẦN LÀM QUEN

Unity Editor, Visual Studio, Git, Blender. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- Unity Intern, Junior Game Developer
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

05 / Game Developer

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 GAME LOOP

Học khái niệm và cách sử dụng Game loop trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 COLLISION

Học khái niệm và cách sử dụng collision trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 ANIMATION

Học khái niệm và cách sử dụng animation trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 AI ĐƠN GIẢN

Học khái niệm và cách sử dụng AI đơn giản trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 TỐI ƯU FPS

Học khái niệm và cách sử dụng tối ưu FPS trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 GAME LOOP

Học khái niệm và cách sử dụng Game loop trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 / Game Developer

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | C#

Học và ứng dụng C#; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | UNITY 2D

Học và ứng dụng Unity 2D; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | PHYSICS

Học và ứng dụng physics; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | UI/GAMEPLAY

Học và ứng dụng UI/gameplay; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | BUILD

Học và ứng dụng build; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | POLISH

Học và ứng dụng polish; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

05 / Game Developer

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • GAME PLATFORMER 2D

Bài toán: Xây dựng game platformer 2d với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • GAME GIẢI ĐỐ

Bài toán: Xây dựng game giải đố với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • GAME SINH TỒN MINI

Bài toán: Xây dựng game sinh tồn mini với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

05 / Game Developer

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: Game loop, collision, animation.
- Biết sử dụng các công cụ chính: Unity Editor, Visual Studio, Git.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Game Developer, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng C# như thế nào trong dự án?
3. Khi Game loop gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

06 / QA Manual Tester

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm QA Manual Tester tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Test case, bug report, SDLC, SQL cơ bản. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Test case, bug report, SDLC, SQL cơ bản; Equivalence partitioning, boundary value, severity, priority, API test.

CÔNG CỤ CẦN LÀM QUEN

Jira, TestRail, Postman, Chrome DevTools. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- QA Intern, Manual Tester
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

06 / QA Manual Tester

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 EQUIVALENCE PARTITIONING

Học khái niệm và cách sử dụng Equivalence partitioning trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 BOUNDARY VALUE

Học khái niệm và cách sử dụng boundary value trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 SEVERITY

Học khái niệm và cách sử dụng severity trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 PRIORITY

Học khái niệm và cách sử dụng priority trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 API TEST

Học khái niệm và cách sử dụng API test trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 EQUIVALENCE PARTITIONING

Học khái niệm và cách sử dụng Equivalence partitioning trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 / QA Manual Tester

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | KIẾN THỨC PHẦN MỀM

Học và ứng dụng Kiến thức phần mềm; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | TEST DESIGN

Học và ứng dụng test design; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | BUG REPORT

Học và ứng dụng bug report; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | API

Học và ứng dụng API; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | SQL

Học và ứng dụng SQL; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | DỰ ÁN

Học và ứng dụng dự án; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

06 / QA Manual Tester

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • BỘ TEST WEBSITE BÁN HÀNG

Bài toán: Xây dựng bộ test website bán hàng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • KIỂM THỬ FORM ĐĂNG KÝ

Bài toán: Xây dựng kiểm thử form đăng ký với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • REGRESSION CHO HỆ THỐNG TUYỂN DỤNG

Bài toán: Xây dựng regression cho hệ thống tuyển dụng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

06 / QA Manual Tester

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: Equivalence partitioning, boundary value, severity.
- Biết sử dụng các công cụ chính: Jira, TestRail, Postman.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn QA Manual Tester, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Test case như thế nào trong dự án?
3. Khi Equivalence partitioning gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

07 / Automation QA Engineer

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Automation QA Engineer tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Python hoặc JavaScript, Playwright, API, CI. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Python hoặc JavaScript, Playwright, API, CI; Locator, assertion, fixture, mock, flaky test, báo cáo CI.

CÔNG CỤ CẦN LÀM QUEN

Playwright, pytest, GitHub Actions, Postman. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- Automation QA Intern, Junior SDET
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

07 / Automation QA Engineer

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 LOCATOR

Học khái niệm và cách sử dụng Locator trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 ASSERTION

Học khái niệm và cách sử dụng assertion trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 FIXTURE

Học khái niệm và cách sử dụng fixture trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 MOCK

Học khái niệm và cách sử dụng mock trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 FLAKY TEST

Học khái niệm và cách sử dụng flaky test trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 BÁO CÁO CI

Học khái niệm và cách sử dụng báo cáo CI trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

07 / Automation QA Engineer

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | MANUAL QA

Học và ứng dụng Manual QA; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | CODE

Học và ứng dụng code; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | PLAYWRIGHT

Học và ứng dụng Playwright; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | API AUTOMATION

Học và ứng dụng API automation; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | CI

Học và ứng dụng CI; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | FRAMEWORK

Học và ứng dụng framework; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

07 / Automation QA Engineer

3 dự án tăng dần độ khó

DỰ ÁN 1 • BỘ UI AUTOMATION TRANG ĐĂNG NHẬP

Bài toán: Xây dựng bộ ui automation trang đăng nhập với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • API REGRESSION SUITE

Bài toán: Xây dựng api regression suite với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • CI CHẠY TEST TỰ ĐỘNG

Bài toán: Xây dựng ci chạy test tự động với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

07 / Automation QA Engineer

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: Locator, assertion, fixture.
- Biết sử dụng các công cụ chính: Playwright, pytest, GitHub Actions.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Automation QA Engineer, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Python hoặc JavaScript như thế nào trong dự án?
3. Khi Locator gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

08 / Business Analyst IT

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Business Analyst IT tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng BPMN, UML, SQL căn bản, Agile, viết tài liệu. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

BPMN, UML, SQL căn bản, Agile, viết tài liệu; Elicitation, user story, acceptance criteria, process modeling.

CÔNG CỤ CẦN LÀM QUEN

Figma, Draw.io, Jira, Confluence, Excel. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- IT BA Intern, Junior Business Analyst
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

08 / Business Analyst IT

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 ELICITATION

Học khái niệm và cách sử dụng Elicitation trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 USER STORY

Học khái niệm và cách sử dụng user story trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 ACCEPTANCE CRITERIA

Học khái niệm và cách sử dụng acceptance criteria trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 PROCESS MODELING

Học khái niệm và cách sử dụng process modeling trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 ELICITATION

Học khái niệm và cách sử dụng Elicitation trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 USER STORY

Học khái niệm và cách sử dụng user story trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

08 / Business Analyst IT

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | NGHIỆP VỤ

Học và ứng dụng Nghiệp vụ; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | YÊU CẦU

Học và ứng dụng yêu cầu; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | BPMN/UML

Học và ứng dụng BPMN/UML; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | WIREFRAME

Học và ứng dụng wireframe; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | BACKLOG

Học và ứng dụng backlog; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | CASE STUDY

Học và ứng dụng case study; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

08 / Business Analyst IT

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • BRD HỆ THỐNG TUYỂN DỤNG

Bài toán: Xây dựng brd hệ thống tuyển dụng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • USE CASE ĐẶT PHÒNG

Bài toán: Xây dựng use case đặt phòng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • PROTOTYPE ỨNG DỤNG HỌC TẬP

Bài toán: Xây dựng prototype ứng dụng học tập với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

08 / Business Analyst IT

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: Elicitation, user story, acceptance criteria.
- Biết sử dụng các công cụ chính: Figma, Draw.io, Jira.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Business Analyst IT, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng BPMN như thế nào trong dự án?
3. Khi Elicitation gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

09 / UI/UX Designer

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm UI/UX Designer tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Figma, typography, layout, UX research. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Figma, typography, layout, UX research; User flow, wireframe, prototype, usability, accessibility.

CÔNG CỤ CẦN LÀM QUEN

Figma, FigJam, Maze, Notion. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- UI/UX Intern, Product Design Intern
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

09 / UI/UX Designer

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 USER FLOW

Học khái niệm và cách sử dụng User flow trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 WIREFRAME

Học khái niệm và cách sử dụng wireframe trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 PROTOTYPE

Học khái niệm và cách sử dụng prototype trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 USABILITY

Học khái niệm và cách sử dụng usability trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 ACCESSIBILITY

Học khái niệm và cách sử dụng accessibility trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 USER FLOW

Học khái niệm và cách sử dụng User flow trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

09 / UI/UX Designer

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | NGUYÊN LÝ THIẾT KẾ

Học và ứng dụng Nguyên lý thiết kế; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | FIGMA

Học và ứng dụng Figma; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | NGHIÊN CỨU

Học và ứng dụng nghiên cứu; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | PROTOTYPE

Học và ứng dụng prototype; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | TEST

Học và ứng dụng test; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | CASE STUDY

Học và ứng dụng case study; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

09 / UI/UX Designer

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • THIẾT KẾ APP QUẢN LÝ CHI TIÊU

Bài toán: Xây dựng thiết kế app quản lý chi tiêu với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • REDESIGN TRANG TUYỂN DỤNG

Bài toán: Xây dựng redesign trang tuyển dụng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • DESIGN SYSTEM MINI

Bài toán: Xây dựng design system mini với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

09 / UI/UX Designer

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: User flow, wireframe, prototype.
- Biết sử dụng các công cụ chính: Figma, FigJam, Maze.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn UI/UX Designer, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Figma như thế nào trong dự án?
3. Khi User flow gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

10 / Data Analyst

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Data Analyst tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Excel, SQL, Power BI, Python cơ bản. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Excel, SQL, Power BI, Python cơ bản; Cleaning, join, KPI, thống kê mô tả, storytelling.

CÔNG CỤ CẦN LÀM QUEN

Excel, Power BI, PostgreSQL, Jupyter. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- Data Analyst Intern, BI Analyst Intern
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

10 / Data Analyst

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 CLEANING

Học khái niệm và cách sử dụng Cleaning trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 JOIN

Học khái niệm và cách sử dụng join trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 KPI

Học khái niệm và cách sử dụng KPI trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 THỐNG KÊ MÔ TẢ

Học khái niệm và cách sử dụng thống kê mô tả trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 STORYTELLING

Học khái niệm và cách sử dụng storytelling trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 CLEANING

Học khái niệm và cách sử dụng Cleaning trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

10 / Data Analyst

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | EXCEL

Học và ứng dụng Excel; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | SQL

Học và ứng dụng SQL; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | POWER BI

Học và ứng dụng Power BI; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | PYTHON

Học và ứng dụng Python; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | PHÂN TÍCH

Học và ứng dụng phân tích; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | PORTFOLIO

Học và ứng dụng portfolio; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

10 / Data Analyst

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • DASHBOARD DOANH THU

Bài toán: Xây dựng dashboard doanh thu với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • PHÂN TÍCH TỶ LỆ NGHỈ VIỆC

Bài toán: Xây dựng phân tích tỷ lệ nghỉ việc với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • PHÂN TÍCH HÀNH VI KHÁCH HÀNG

Bài toán: Xây dựng phân tích hành vi khách hàng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

10 / Data Analyst

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: Cleaning, join, KPI.
- Biết sử dụng các công cụ chính: Excel, Power BI, PostgreSQL.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Data Analyst, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Excel như thế nào trong dự án?
3. Khi Cleaning gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

11 / Data Engineer

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Data Engineer tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Python, SQL, ETL/ELT, data warehouse. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Python, SQL, ETL/ELT, data warehouse; Data modeling, batch, orchestration, quality, partitioning.

CÔNG CỤ CẦN LÀM QUEN

PostgreSQL, Airflow, dbt, Docker. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- Data Engineer Intern, Junior Data Engineer
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

11 / Data Engineer

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 DATA MODELING

Học khái niệm và cách sử dụng Data modeling trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 BATCH

Học khái niệm và cách sử dụng batch trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 ORCHESTRATION

Học khái niệm và cách sử dụng orchestration trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 QUALITY

Học khái niệm và cách sử dụng quality trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 PARTITIONING

Học khái niệm và cách sử dụng partitioning trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 DATA MODELING

Học khái niệm và cách sử dụng Data modeling trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

11 / Data Engineer

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | PYTHON/SQL

Học và ứng dụng Python/SQL; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | ETL

Học và ứng dụng ETL; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | WAREHOUSE

Học và ứng dụng warehouse; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | AIRFLOW

Học và ứng dụng Airflow; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | DBT

Học và ứng dụng dbt; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | MONITORING

Học và ứng dụng monitoring; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

11 / Data Engineer

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • PIPELINE DỮ LIỆU ĐƠN HÀNG

Bài toán: Xây dựng pipeline dữ liệu đơn hàng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • DATA WAREHOUSE BÁN LẺ

Bài toán: Xây dựng data warehouse bán lẻ với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • ETL LỊCH CHẠY HẰNG NGÀY

Bài toán: Xây dựng etl lịch chạy hằng ngày với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

11 / Data Engineer

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: Data modeling, batch, orchestration.
- Biết sử dụng các công cụ chính: PostgreSQL, Airflow, dbt.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Data Engineer, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Python như thế nào trong dự án?
3. Khi Data modeling gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

12 / Data Scientist

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Data Scientist tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Python, xác suất thống kê, machine learning. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Python, xác suất thống kê, machine learning; EDA, feature engineering, cross-validation, metric, leakage.

CÔNG CỤ CẦN LÀM QUEN

pandas, scikit-learn, Jupyter, matplotlib. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- Data Science Intern, Junior Data Scientist
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

12 / Data Scientist

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 EDA

Học khái niệm và cách sử dụng EDA trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 FEATURE ENGINEERING

Học khái niệm và cách sử dụng feature engineering trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 CROSS-VALIDATION

Học khái niệm và cách sử dụng cross-validation trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 METRIC

Học khái niệm và cách sử dụng metric trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 LEAKAGE

Học khái niệm và cách sử dụng leakage trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 EDA

Học khái niệm và cách sử dụng EDA trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

12 / Data Scientist

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | PYTHON

Học và ứng dụng Python; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | THỐNG KÊ

Học và ứng dụng thống kê; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | EDA

Học và ứng dụng EDA; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | ML

Học và ứng dụng ML; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | ĐÁNH GIÁ

Học và ứng dụng đánh giá; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | TRIỂN KHAI DEMO

Học và ứng dụng triển khai demo; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

12 / Data Scientist

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • DỰ ĐOÁN CHURN

Bài toán: Xây dựng dự đoán churn với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • PHÂN NHÓM KHÁCH HÀNG

Bài toán: Xây dựng phân nhóm khách hàng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • DỰ BÁO NHU CẦU BÁN HÀNG

Bài toán: Xây dựng dự báo nhu cầu bán hàng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

12 / Data Scientist

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: EDA, feature engineering, cross-validation.
- Biết sử dụng các công cụ chính: pandas, scikit-learn, Jupyter.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Data Scientist, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Python như thế nào trong dự án?
3. Khi EDA gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

13 / AI/ML Engineer

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm AI/ML Engineer tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Python, đại số tuyến tính, ML, API. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Python, đại số tuyến tính, ML, API; Training, inference, embeddings, evaluation, model serving.

CÔNG CỤ CẦN LÀM QUEN

PyTorch, scikit-learn, FastAPI, MLflow. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- ML Engineer Intern, AI Engineer Intern
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

13 / AI/ML Engineer

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 TRAINING

Học khái niệm và cách sử dụng Training trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 INFERENCE

Học khái niệm và cách sử dụng inference trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 EMBEDDINGS

Học khái niệm và cách sử dụng embeddings trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 EVALUATION

Học khái niệm và cách sử dụng evaluation trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 MODEL SERVING

Học khái niệm và cách sử dụng model serving trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 TRAINING

Học khái niệm và cách sử dụng Training trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

13 / AI/ML Engineer

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | PYTHON/TOÁN

Học và ứng dụng Python/toán; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | ML

Học và ứng dụng ML; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | DEEP LEARNING

Học và ứng dụng deep learning; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | SERVING

Học và ứng dụng serving; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | MONITORING

Học và ứng dụng monitoring; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | PORTFOLIO

Học và ứng dụng portfolio; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

13 / AI/ML Engineer

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • PHÂN LOẠI ẢNH

Bài toán: Xây dựng phân loại ảnh với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • HỆ THỐNG GỢI Ý

Bài toán: Xây dựng hệ thống gợi ý với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • API DỰ ĐOÁN CÓ THEO DÕI MODEL

Bài toán: Xây dựng api dự đoán có theo dõi model với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

13 / AI/ML Engineer

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: Training, inference, embeddings.
- Biết sử dụng các công cụ chính: PyTorch, scikit-learn, FastAPI.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn AI/ML Engineer, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Python như thế nào trong dự án?
3. Khi Training gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

14 / Generative AI Engineer

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Generative AI Engineer tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Python, LLM API, RAG, vector database. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Python, LLM API, RAG, vector database; Prompt, chunking, retrieval, hallucination, evaluation, guardrails.

CÔNG CỤ CẦN LÀM QUEN

LangChain, FAISS, FastAPI, Docker. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- GenAI Intern, LLM Application Developer
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

14 / Generative AI Engineer

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 PROMPT

Học khái niệm và cách sử dụng Prompt trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 CHUNKING

Học khái niệm và cách sử dụng chunking trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 RETRIEVAL

Học khái niệm và cách sử dụng retrieval trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 HALLUCINATION

Học khái niệm và cách sử dụng hallucination trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 EVALUATION

Học khái niệm và cách sử dụng evaluation trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 GUARDRAILS

Học khái niệm và cách sử dụng guardrails trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

14 / Generative AI Engineer

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | PYTHON/API

Học và ứng dụng Python/API; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | LLM

Học và ứng dụng LLM; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | EMBEDDING

Học và ứng dụng embedding; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | RAG

Học và ứng dụng RAG; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | EVAL

Học và ứng dụng eval; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | DEPLOY

Học và ứng dụng deploy; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

14 / Generative AI Engineer

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • CHATBOT HỎI ĐÁP TÀI LIỆU

Bài toán: Xây dựng chatbot hỏi đáp tài liệu với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • TRỢ LÝ TÌM VIỆC RAG

Bài toán: Xây dựng trợ lý tìm việc rag với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • BỘ ĐÁNH GIÁ CÂU TRẢ LỜI AI

Bài toán: Xây dựng bộ đánh giá câu trả lời ai với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

14 / Generative AI Engineer

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: Prompt, chunking, retrieval.
- Biết sử dụng các công cụ chính: LangChain, FAISS, FastAPI.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Generative AI Engineer, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Python như thế nào trong dự án?
3. Khi Prompt gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

15 / Cybersecurity Analyst

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Cybersecurity Analyst tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Mạng máy tính, Linux, HTTP, security basics. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Mạng máy tính, Linux, HTTP, security basics; CIA, OWASP Top 10, log analysis, incident response, ethics.

CÔNG CỤ CẦN LÀM QUEN

Wireshark, Burp Suite Community, SIEM lab. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- SOC Analyst Intern, Security Analyst Intern
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

15 / Cybersecurity Analyst

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 CIA

Học khái niệm và cách sử dụng CIA trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 OWASP TOP 10

Học khái niệm và cách sử dụng OWASP Top 10 trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 LOG ANALYSIS

Học khái niệm và cách sử dụng log analysis trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 INCIDENT RESPONSE

Học khái niệm và cách sử dụng incident response trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 ETHICS

Học khái niệm và cách sử dụng ethics trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 CIA

Học khái niệm và cách sử dụng CIA trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

15 / Cybersecurity Analyst

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | MẠNG/LINUX

Học và ứng dụng Mạng/Linux; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | WEB SECURITY

Học và ứng dụng web security; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | LAB HỢP PHÁP

Học và ứng dụng lab hợp pháp; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | LOG

Học và ứng dụng log; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | SOC

Học và ứng dụng SOC; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | PORTFOLIO

Học và ứng dụng portfolio; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

15 / Cybersecurity Analyst

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • PHÂN TÍCH LOG ĐĂNG NHẬP

Bài toán: Xây dựng phân tích log đăng nhập với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • BÁO CÁO LỖ HỔNG LAB

Bài toán: Xây dựng báo cáo lỗ hổng lab với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • DASHBOARD CẢNH BÁO

Bài toán: Xây dựng dashboard cảnh báo với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

15 / Cybersecurity Analyst

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: CIA, OWASP Top 10, log analysis.
- Biết sử dụng các công cụ chính: Wireshark, Burp Suite Community, SIEM lab.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Cybersecurity Analyst, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Mạng máy tính như thế nào trong dự án?
3. Khi CIA gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

16 / DevOps Engineer

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm DevOps Engineer tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Linux, Bash, Git, Docker, CI/CD, cloud. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Linux, Bash, Git, Docker, CI/CD, cloud; Networking, deployment, IaC, secrets, observability, rollback.

CÔNG CỤ CẦN LÀM QUEN

Docker, GitHub Actions, Terraform, Prometheus. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- DevOps Intern, Junior DevOps Engineer
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

16 / DevOps Engineer

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 NETWORKING

Học khái niệm và cách sử dụng Networking trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 DEPLOYMENT

Học khái niệm và cách sử dụng deployment trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 IAC

Học khái niệm và cách sử dụng IaC trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 SECRETS

Học khái niệm và cách sử dụng secrets trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 OBSERVABILITY

Học khái niệm và cách sử dụng observability trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 ROLLBACK

Học khái niệm và cách sử dụng rollback trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

16 / DevOps Engineer

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | LINUX

Học và ứng dụng Linux; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | BASH/GIT

Học và ứng dụng Bash/Git; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | DOCKER

Học và ứng dụng Docker; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | CI/CD

Học và ứng dụng CI/CD; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | CLOUD

Học và ứng dụng cloud; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | IAC/MONITORING

Học và ứng dụng IaC/monitoring; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

16 / DevOps Engineer

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • CI/CD ỨNG DỤNG WEB

Bài toán: Xây dựng ci/cd ứng dụng web với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • CONTAINER HÓA API

Bài toán: Xây dựng container hóa api với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • MONITORING HỆ THỐNG DEMO

Bài toán: Xây dựng monitoring hệ thống demo với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

16 / DevOps Engineer

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: Networking, deployment, IaC.
- Biết sử dụng các công cụ chính: Docker, GitHub Actions, Terraform.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn DevOps Engineer, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Linux như thế nào trong dự án?
3. Khi Networking gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

17 / Cloud Engineer

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Cloud Engineer tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Linux, networking, cloud fundamentals, IaC. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Linux, networking, cloud fundamentals, IaC; IAM, VPC, compute, storage, cost, high availability.

CÔNG CỤ CẦN LÀM QUEN

AWS hoặc Azure, Terraform, Docker. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- Cloud Support Intern, Junior Cloud Engineer
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

17 / Cloud Engineer

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 IAM

Học khái niệm và cách sử dụng IAM trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 VPC

Học khái niệm và cách sử dụng VPC trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 COMPUTE

Học khái niệm và cách sử dụng compute trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 STORAGE

Học khái niệm và cách sử dụng storage trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 COST

Học khái niệm và cách sử dụng cost trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 HIGH AVAILABILITY

Học khái niệm và cách sử dụng high availability trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

17 / Cloud Engineer

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | MẠNG/LINUX

Học và ứng dụng Mạng/Linux; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | CLOUD

Học và ứng dụng cloud; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | IAM/NETWORK

Học và ứng dụng IAM/network; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | DEPLOY

Học và ứng dụng deploy; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | IAC

Học và ứng dụng IaC; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | TỐI ƯU

Học và ứng dụng tối ưu; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

17 / Cloud Engineer

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • TRIỂN KHAI WEBSITE TRÊN CLOUD

Bài toán: Xây dựng triển khai website trên cloud với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • KIẾN TRÚC 3 TẦNG

Bài toán: Xây dựng kiến trúc 3 tầng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • BACKUP VÀ CẢNH BÁO

Bài toán: Xây dựng backup và cảnh báo với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

17 / Cloud Engineer

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: IAM, VPC, compute.
- Biết sử dụng các công cụ chính: AWS hoặc Azure, Terraform, Docker.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Cloud Engineer, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Linux như thế nào trong dự án?
3. Khi IAM gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

18 / System Administrator

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm System Administrator tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Windows/Linux, TCP/IP, DNS, backup. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Windows/Linux, TCP/IP, DNS, backup; User permission, service, patch, backup, incident handling.

CÔNG CỤ CẦN LÀM QUEN

Windows Server lab, Ubuntu, VMware, Zabbix. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- IT Support Intern, Junior System Admin
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

18 / System Administrator

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 USER PERMISSION

Học khái niệm và cách sử dụng User permission trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 SERVICE

Học khái niệm và cách sử dụng service trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 PATCH

Học khái niệm và cách sử dụng patch trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 BACKUP

Học khái niệm và cách sử dụng backup trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 INCIDENT HANDLING

Học khái niệm và cách sử dụng incident handling trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 USER PERMISSION

Học khái niệm và cách sử dụng User permission trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

18 / System Administrator

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | OS

Học và ứng dụng OS; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | MẠNG

Học và ứng dụng mạng; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | DỊCH VỤ

Học và ứng dụng dịch vụ; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | SCRIPT

Học và ứng dụng script; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | MONITORING

Học và ứng dụng monitoring; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | VẬN HÀNH LAB

Học và ứng dụng vận hành lab; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

18 / System Administrator

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • MÁY CHỦ FILE NỘI BỘ

Bài toán: Xây dựng máy chủ file nội bộ với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • DNS LAB

Bài toán: Xây dựng dns lab với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • HỆ THỐNG SAO LƯU VÀ GIÁM SÁT

Bài toán: Xây dựng hệ thống sao lưu và giám sát với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

18 / System Administrator

05 / Ứng tuyển & tự đánh giá

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: User permission, service, patch.
- Biết sử dụng các công cụ chính: Windows Server lab, Ubuntu, VMware.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn System Administrator, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Windows/Linux như thế nào trong dự án?
3. Khi User permission gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

19 / Embedded/IoT Engineer

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Embedded/IoT Engineer tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng C/C++, điện tử cơ bản, vi điều khiển. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

C/C++, điện tử cơ bản, vi điều khiển; GPIO, interrupts, UART/I2C, sensor, MQTT, tiết kiệm điện.

CÔNG CỤ CẦN LÀM QUEN

Arduino IDE, ESP32, PlatformIO, MQTT. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- Embedded Intern, IoT Developer Intern
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

19 / Embedded/IoT Engineer

02 / Kiến thức và kỹ năng cốt lõi

6 năng lực cần xây dựng

01 GPIO

Học khái niệm và cách sử dụng GPIO trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 INTERRUPTS

Học khái niệm và cách sử dụng interrupts trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 UART/I2C

Học khái niệm và cách sử dụng UART/I2C trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 SENSOR

Học khái niệm và cách sử dụng sensor trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 MQTT

Học khái niệm và cách sử dụng MQTT trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 TIẾT KIỆM ĐIỆN

Học khái niệm và cách sử dụng tiết kiệm điện trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

19 / Embedded/IoT Engineer

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | C

Học và ứng dụng C; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | ĐIỆN TỬ

Học và ứng dụng điện tử; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | ESP32

Học và ứng dụng ESP32; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | GIAO TIẾP

Học và ứng dụng giao tiếp; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | IOT CLOUD

Học và ứng dụng IoT cloud; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | SẢN PHẨM

Học và ứng dụng sản phẩm; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

19 / Embedded/IoT Engineer

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • ĐÈN THÔNG MINH

Bài toán: Xây dựng đèn thông minh với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • CẢM BIẾN NHIỆT ĐỘ

Bài toán: Xây dựng cảm biến nhiệt độ với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • HỆ THỐNG TƯỚI CÂY TỰ ĐỘNG

Bài toán: Xây dựng hệ thống tưới cây tự động với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

19 / Embedded/IoT Engineer

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: GPIO, interrupts, UART/I2C.
- Biết sử dụng các công cụ chính: Arduino IDE, ESP32, PlatformIO.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Embedded/IoT Engineer, thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng C/C++ như thế nào trong dự án?
3. Khi GPIO gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.

20 / Product Manager (Tech)

01 / Bức tranh nghề nghiệp

NGHỀ NÀY GIẢI QUYẾT VẤN ĐỀ GÌ?

Người làm Product Manager (Tech) tạo ra hoặc cải thiện sản phẩm và hệ thống số bằng cách vận dụng Product discovery, metrics, Agile, UX, SQL căn bản. Giá trị công việc được đánh giá qua sản phẩm chạy được, độ tin cậy, trải nghiệm người dùng và khả năng cộng tác.

Một ngày làm việc điển hình

- Đọc yêu cầu, xác định mục tiêu, đầu vào, đầu ra và tiêu chí hoàn thành.
- Thực hiện công việc chuyên môn; kiểm tra kết quả với dữ liệu hoặc tình huống thực tế.
- Trao đổi tiến độ, nhận phản hồi, sửa lỗi và ghi chép quyết định quan trọng.
- Bàn giao bằng chứng rõ ràng: tài liệu, mã nguồn, bản demo hoặc báo cáo.

Công nghệ / kiến thức thường gặp

NỀN TẢNG CẦN NHẬN DIỆN

Product discovery, metrics, Agile, UX, SQL căn bản; Problem framing, prioritization, funnel, stakeholder, release.

CÔNG CỤ CẦN LÀM QUEN

Notion, Jira, Figma, Excel, Amplitude demo. Không cần biết tất cả ngay từ đầu; ưu tiên công cụ dùng trong dự án đầu tiên.

Vị trí đầu vào có thể tìm kiếm

- Product Intern, Associate Product Manager
- Lưu ý: tên vị trí và tiêu chí tuyển dụng thay đổi theo doanh nghiệp; đọc JD cụ thể trước khi ứng tuyển.

20 / Product Manager (Tech)

6 năng lực cần xây dựng

01 PROBLEM FRAMING

Học khái niệm và cách sử dụng Problem framing trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

02 PRIORITIZATION

Học khái niệm và cách sử dụng prioritization trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

03 FUNNEL

Học khái niệm và cách sử dụng funnel trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

04 STAKEHOLDER

Học khái niệm và cách sử dụng stakeholder trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

05 RELEASE

Học khái niệm và cách sử dụng release trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

06 PROBLEM FRAMING

Học khái niệm và cách sử dụng Problem framing trong một bài toán nhỏ. Tự tạo ví dụ, giải thích lựa chọn của mình, ghi lại lỗi gặp phải và kiểm chứng bằng kết quả cụ thể.

20 / Product Manager (Tech)

03 / Lộ trình 12 tháng từ con số 0

CƯỜNG ĐỘ GỢI Ý

8–12 giờ/tuần: 30% học khái niệm, 50% thực hành, 20% ôn tập và ghi chép. Mỗi tháng tạo một đầu ra có thể chia sẻ.

THÁNG 01–02 | SẢN PHẨM

Học và ứng dụng Sản phẩm; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 03–04 | NGHIÊN CỨU

Học và ứng dụng nghiên cứu; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 05–06 | PRD

Học và ứng dụng PRD; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 07–08 | METRICS

Học và ứng dụng metrics; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 09–10 | ROADMAP

Học và ứng dụng roadmap; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

THÁNG 11–12 | CASE STUDY

Học và ứng dụng case study; làm 8 bài tập ngắn và 1 mini-project. Viết README, tự giải thích cách làm, thử tình huống lỗi và cải tiến sau phản hồi.

20 / Product Manager (Tech)

04 / Dự án thực hành & portfolio

3 dự án tăng dần độ khó

DỰ ÁN 1 • PRD TÍNH NĂNG TUYỂN DỤNG

Bài toán: Xây dựng prd tính năng tuyển dụng với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 2 • ROADMAP APP HỌC TẬP

Bài toán: Xây dựng roadmap app học tập với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

DỰ ÁN 3 • A/B TEST GIẢ LẬP

Bài toán: Xây dựng a/b test giả lập với phạm vi đủ nhỏ để hoàn thành.
Đầu ra: mô tả yêu cầu, bản demo hoặc ảnh chụp kết quả, dữ liệu thử nghiệm, tài liệu hướng dẫn.
Tiêu chí đạt: chạy lại được, có kiểm tra tình huống lỗi, giải thích được quyết định kỹ thuật và nêu được hạn chế.
Nâng cấp: bổ sung đo lường, tự động hóa hoặc trải nghiệm người dùng phù hợp.

Cấu trúc portfolio nên có

- Tên dự án và vấn đề giải quyết; đối tượng sử dụng và phạm vi.
- Công nghệ/công cụ, kiến trúc hoặc quy trình; phần việc tự thực hiện.
- Hướng dẫn chạy hoặc đường dẫn demo; ảnh màn hình và kết quả kiểm thử.
- Khó khăn, cách khắc phục, bài học và kế hoạch cải tiến.

20 / Product Manager (Tech)

Checklist trước khi gửi CV

- Có ít nhất 2 dự án hoàn thiện, không chỉ sao chép tutorial.
- Giải thích được kiến thức nền: Problem framing, prioritization, funnel.
- Biết sử dụng các công cụ chính: Notion, Jira, Figma.
- Có CV 1 trang, portfolio rõ vai trò và liên kết truy cập được.
- Đọc JD và đánh dấu yêu cầu bắt buộc, ưu tiên, có thể học bổ sung.
- Tập trình bày dự án trong 3 phút: vấn đề → giải pháp → kết quả.

Câu hỏi tự luyện phỏng vấn

1. Tại sao chọn Product Manager (Tech), thay vì một vị trí CNTT khác?
2. Bạn đã sử dụng Product discovery như thế nào trong dự án?
3. Khi Problem framing gặp lỗi hoặc kết quả bất thường, bạn xử lý ra sao?
4. Hãy kể một lần nhận phản hồi và cải tiến sản phẩm.
5. Nếu có thêm hai tuần, bạn sẽ ưu tiên nâng cấp phần nào và vì sao?

KẾ HOẠCH 7 NGÀY TIẾP THEO

Ngày 1: rà soát kiến thức thiếu. Ngày 2–3: hoàn thiện một tính năng. Ngày 4: kiểm thử và sửa lỗi. Ngày 5: viết README. Ngày 6: cập nhật CV/portfolio. Ngày 7: mock interview và ghi lại 3 điểm cần cải thiện.

NGUỒN HỌC ĐỂ BẮT ĐẦU

Ưu tiên tài liệu chính thức của công cụ trong nghề; tham khảo MDN Web Docs, freeCodeCamp, Microsoft Learn, Google Developers, AWS Skill Builder, Kaggle Learn, OWASP và tài liệu GitHub. Kiểm tra cập nhật và điều khoản sử dụng trước khi học.